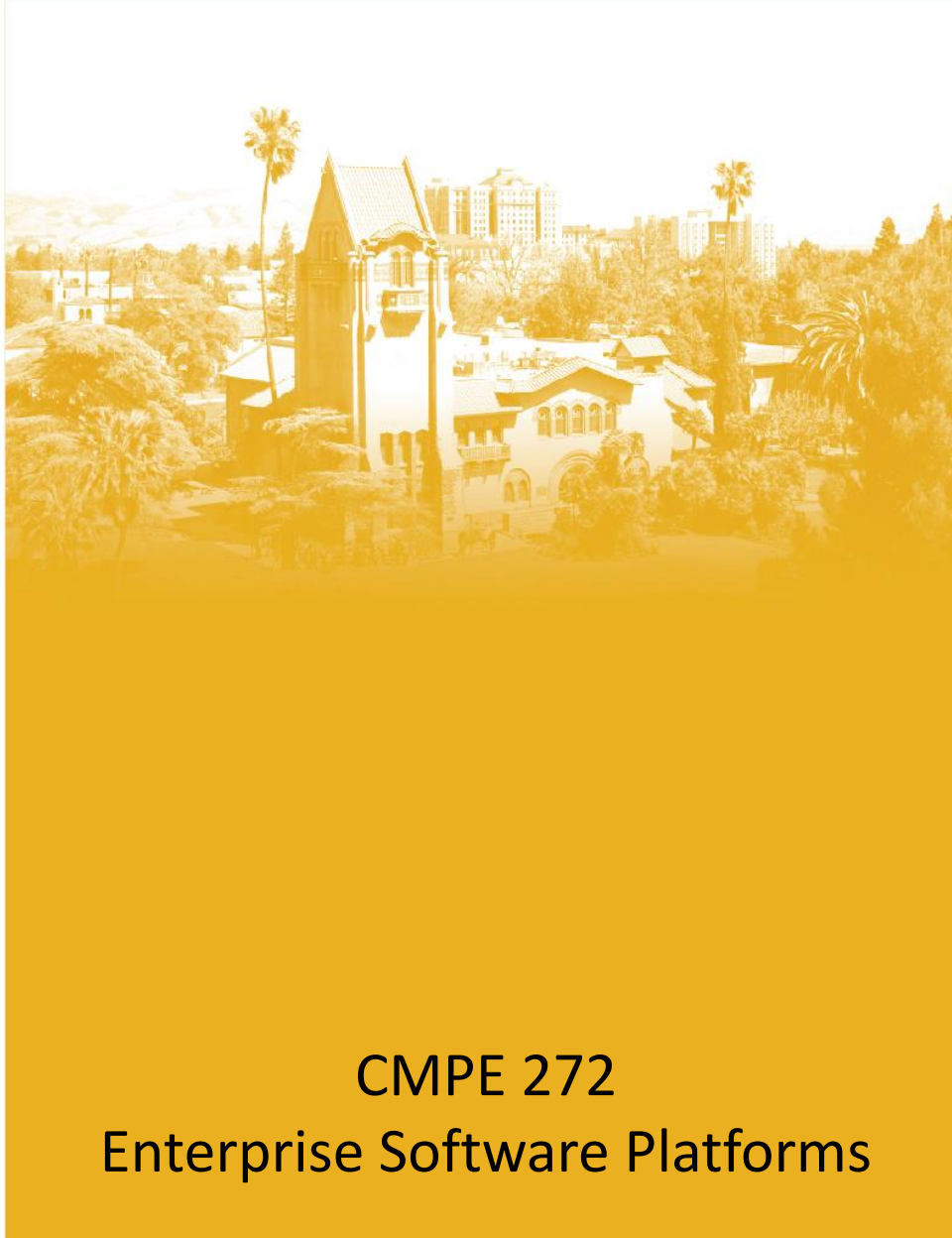
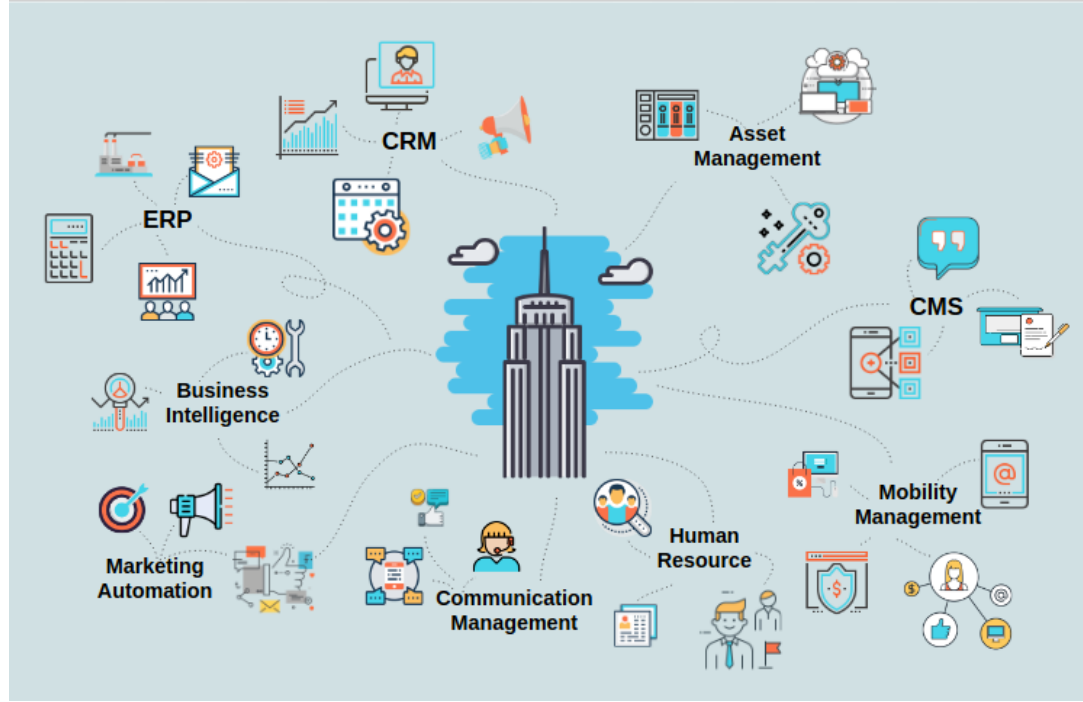




CMPE 272

Enterprise Software Platforms



CMPE 272

Enterprise Software Platforms

Instructor: Andrew Bond

About the Course

Enterprise Software Platforms

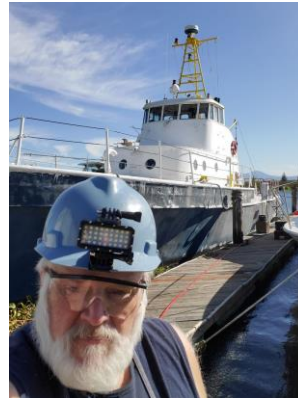
Description

Enterprise software platforms: the technologies, architectures and practices used to build and operate software at organizational scale — from infrastructure and application frameworks through data platforms, enterprise applications, AI integration, security and operations, with a substantial team project.

Prerequisite: Classified graduate standing or graduate advisor consent.

About The Instructor

- Andrew Bond is a Lead Consultant at AT&T Consulting
- Graduate of SJSU Cloud MSSE Program (Cohort #1, 2012)
- IEEE Senior Member, joined in 1990
- What I do for fun:



Student Intros

- Name
- Major
- Technical Interests
- Something about you

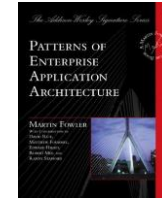
Course Learning Objectives

	Description
CLO 1	Describe the enterprise software stack end to end and how its layers interact
CLO 2	Evaluate application frameworks, APIs and integration patterns against enterprise requirements
CLO 3	Compare cloud-native options (containers, serverless, mesh) on cost, complexity, operations
CLO 4	Explain the role of ERP, SCM and CRM systems and how they are extended and integrated
CLO 5	Model a business process and compose services to implement it
CLO 6	Select a datastore for an access pattern and design a data pipeline over it

Course Texts

Required Texts

- Patterns of Enterprise Application Architecture (~\$20 on [Amazon](#))
Author: Martin Fowler ([Catalog of Patterns](#))
- Building Microservices Author: Sam Newman (also ~\$20)



Recommended Texts

- The Pragmatic Programmer: From Journeyman to Master Authors: Andrew Hunt, David Thomas
- Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions Authors: Gregor Hohpe, Bobby Woolf
- Next Generation SOA: A Concise Introduction to Service Technology & Service-Oriented Architecture Author: Thomas Erl
- <http://martinfowler.com>
- <http://www.oreilly.com/design/free/files/designing-for-the-internet-of-things.pdf>
- <http://www.opengroup.org/togaf>
- <https://thenewstack.io/ebooks/>

THE NEW STACK

Handouts

- Lecture slides and other class supplemental materials will be distributed via the Canvas Portal.

Course Accounts & Software Subscriptions (Required)

- [AWS Educate](#)



- Grants for free usage of AWS services
- Labs, tutorials and training on AWS products
- Open source course content provided by top professors and AWS
- Collaboration tools and a community of cloud experts

- [GitHub](#) (*bitbicket, gitlab, etc..*)



- [Cisco DCloud](#) [Create a new account](#)



- [Tableau Student Edition](#)



- [Salesforce Trailhead](#)



<https://www.youtube.com/watch?v=Y8c2le3ammM>

Assignments and Grading Policy

- Grades will be assigned on a curve.
 - Final grades in this course will be based on assignments, project, exams, and class participation.
-
- | | |
|------------------------------|-----|
| • Homework Assignments (7-8) | 22% |
| • In-class Assignments (14) | 10% |
| • 1 Term project | 33% |
| • 1 Midterm | 10% |
| • 1 Final | 25% |

Grading Rules

- **Grading Rules:**
- 100% A+
- 93-99% A
- 90-92% A-
- 87-89% B+
- 83-86% B
- 80-82% B-
- 77-79% C+
- 73-76% C
- 70-72% C-

Academic Integrity

Your commitment, as a student, to learning is evidenced by your enrollment at San Jose State University. The [University Academic Integrity Policy S07-2](http://www.sjsu.edu/senate/docs/S07-2.pdf) at <http://www.sjsu.edu/senate/docs/S07-2.pdf> requires you to be honest in all your academic course work. Faculty members are required to report all infractions to the office of Student Conduct and Ethical Development. The [Student Conduct and Ethical Development website](http://www.sjsu.edu/studentconduct/) is available at <http://www.sjsu.edu/studentconduct/>.

Plagiarism

- San José State University defines plagiarism as the act of representing the work of another as one's own without giving appropriate credit, regardless of how that work was obtained, and submitting it to fulfill academic requirements.
- Plagiarism includes:
 - Knowingly or unknowingly incorporating the **ideas, words, sentences, paragraphs, or parts of**, or the **specific substance** of another's work, without giving appropriate credit, and representing the product as one's own work;
 - Representing another's artistic/scholarly works such as musical compositions, computer programs, **photographs**, paintings, **drawing**, sculptures, or similar works as one's own.

Teams

- Teams of 2-4 people
- Team Name (unique)
- Designated Team Leader
- All assignments will be submitted by Team Leader only

You make half of the assignment and I'll make the other half and then we'll join them together



CMPE-272 Project

Default Project:

Your team works for a company that wants to create an enterprise HR Portal application.

Your manager has provided a [backup of the company's enterprise database](#),

and asks that you use it to demonstrate a Proof-of-Concept application including:

- Single Sign-on (SSO) / AD authentication with SSL/TLS encryption
- Application / Web portal for viewing/browsing (sample) enterprise employee data (with SSO roles)
- [Github](#) (or other repo) integrated into SSO (optional)
- Jenkins, integrated into SSO and Github repo;
- Additional integrations / features / capabilities for higher grade [layered security, social media integration, document repository, Salesforce integration, etc..]

• Project Deliverables:

- Project Plan in .docx format. There's a template in Canvas, or you can make your own
- Project Presentation in .pptx format
- Project Report: .docx format, including design patterns, diagrams, use/test cases, screenshots, etc. There is a template in Canvas
- 15 minute live project presentation, where each team member MUST present/speak for a portion
- Code (checked into github, with references in Project Report: <https://github.com/sjsu-cmpe272>)

Class Schedule

Week	03 Tue / 49 Wed	Topics, Assignments, Deadlines
1	8/25 / 8/19	Introduction and Overview
2	9/1 / 8/26	Enterprise Infrastructure (HW1)
3	9/8 / 9/2	Application Frameworks and APIs (HW2)
4	9/15 / 9/9	Cloud-Native Services
5	9/22 / 9/16	Platform Engineering & the Delivery Lifecycle (HW3)
6	9/29 / 9/23	Enterprise Applications: ERP, SCM and CRM (HW4)
7	10/6 / 9/30	Business Processes, Messaging & Service Composition
8	10/13 / 10/7	Project Abstract Presentations + Midterm Exam
9	10/20 / 10/14	Datastores, Databases and Query Languages (HW5)
10	10/27 / 10/21	Data Engineering and the Modern Data Stack (HW6)
11	11/3 / 10/28	Enterprise AI I — Platforms and MLOps (HW7)
12	11/10 / 11/4	Enterprise AI II — LLM Integration
13	11/17 / 11/18	Enterprise Security, Identity and Supply Chain (HW8)
14	11/24 / async	Resilience, Observability & Monitoring (49: async on Canvas)
15	12/1 / 12/2	Final Project Presentations (last week of instruction)
—	12/15 / 12/9	Final Exam, 5:30 – 7:30 PM (03: Tue 12/15 · 49: Wed 12/9)

Topic Coverage

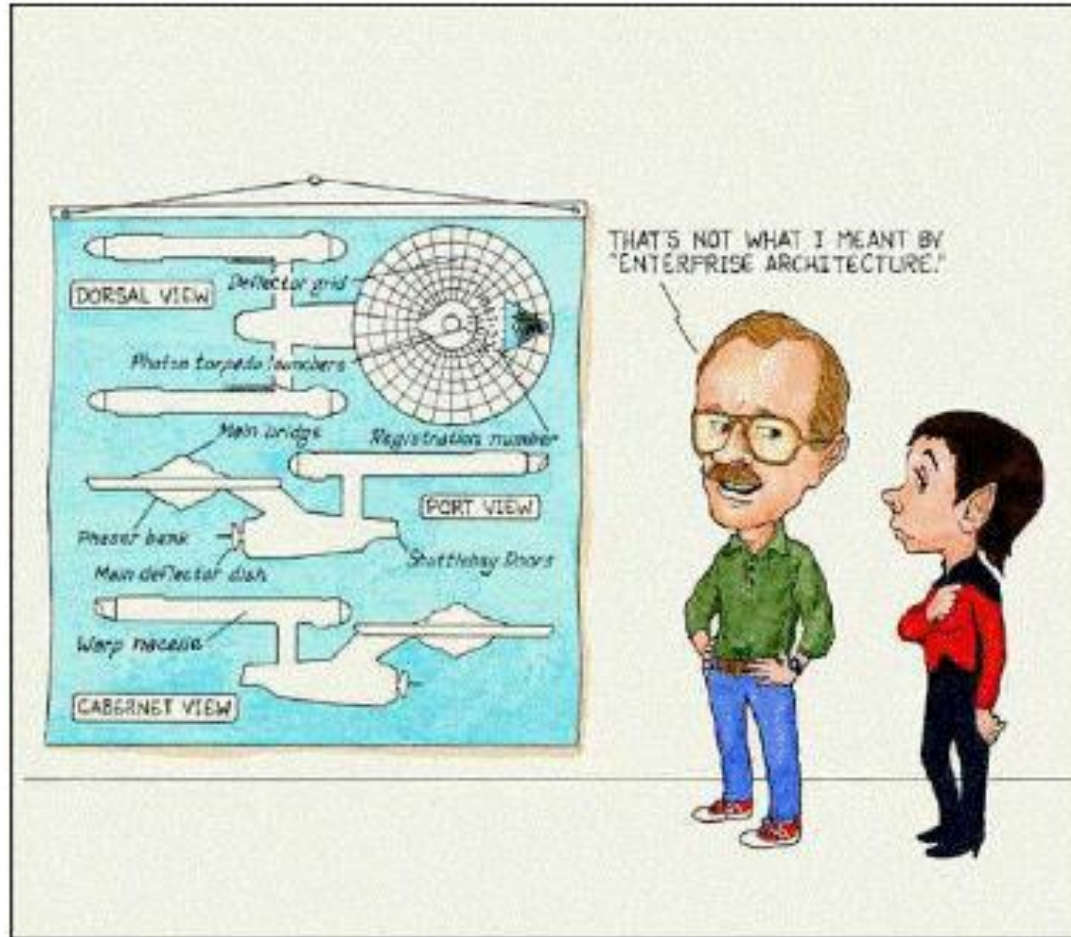
- What do you want to see?
- Feedback is appreciated, I'm flexible

What do I Mean By Enterprise Applications?

Characteristics

- Data Persistence
- Data Access Concurrency
- Data Polymorphism (UI)
- Continuous Delivery and Continuous Integration (CI/CD)
- Reliability, Availability, Scalability (RAS), DR
- Integration, Instrumentation/Telemetry & Manageability
- Data & Information Security
- Authorization, Authentication, Accounting/Auditing (AAA)

What do I mean by Enterprise Architecture?



What is an enterprise?

- **TOGAF** (*The Open Group Architectural Framework*) defines "**enterprise**" as any collection of organizations that has a common set of goals.
- Such as:
 - A government agency
 - A whole corporation
 - A division of a corporation
 - A single department
 - A chain of geographically distant organizations linked together by common ownership

What is an Enterprise Application?

- Examples

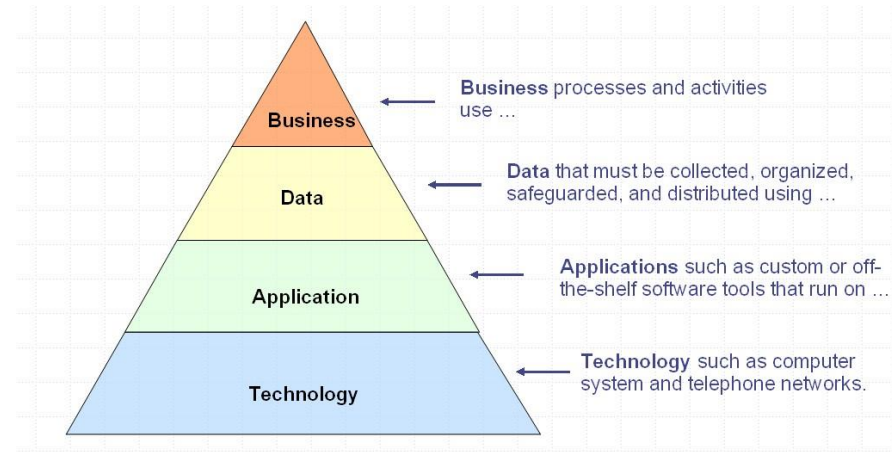
- Payroll
- Patient records
- Shipment tracking
- Credit scoring
- Insurance processing
- Supply chain management
- Accounting
- Customer relationship management
- Foreign trading

Why is Enterprise Architecture Needed?

- **Business Efficiency**
 - Lower operating costs
 - Organizational Agility
 - Lower change management costs
 - Improved business productivity
- **IT Operational Efficiency**
 - Lower software development, support, and maintenance costs
 - Increase application portability
 - Improve IT/Data management and analysis
 - Improved security, reliability and availability of IT systems and data
 - Easier upgrade and exchange of IT system components
- **Better return on existing investment, reduced risk for future investment:**
 - Reduced complexity in the enterprise and IT
 - Maximum return on investment in existing enterprise and IT infrastructure
 - The flexibility to make, buy, or out-source enterprise and IT solutions
 - Reduced the risk in new IT investments and their cost of ownership

Enterprise Architecture Frameworks

- The Zachman Framework for Enterprise Architecture
- The Open Group Architecture Framework (TOGAF)
 - TOGAF Architecture Development Methodology (ADM)
 - TOGAF Architecture Content Framework
 - TOGAF Architecture Capability Framework
- The Role and Use of [EAP](#) Frameworks



“Divides architecture description into domains, layers, or views, and offers models - typically matrices and diagrams - for documenting each view” ([Wikipedia](#))

Why Architecture Matters

- <https://www.youtube.com/watch?v=DngAZyWMGR0>
(10 mins)



[Martin Fowler](#)

"..author, speaker... essentially a loud-mouthed pundit on the topic of software development, primarily for Enterprise Applications".



TOPICS

#1 Enterprise Infrastructure

- Operating environments: Linux, Windows, virtualization
- Containers: Docker, Kubernetes fundamentals
- Enterprise networking: SDN, security, cloud connectivity
- From server administration to managed platforms

#2 Application Frameworks and APIs

- Enterprise application frameworks
- API design and API gateways
- Integration patterns: REST, gRPC, events, legacy SOAP
- Integration as a first-class architectural concern

#3 Cloud-Native Services

- Containers and orchestration
- Serverless and lambda services
- Service mesh — where the complexity is (and is not) repaid
- Sync vs. async patterns; orchestration vs. choreography

#4 Platform Engineering and the Delivery Lifecycle

Internal developer platforms; platform-as-product

Software development models and lifecycles

CI/CD pipelines: Git, Jenkins, GitHub Actions

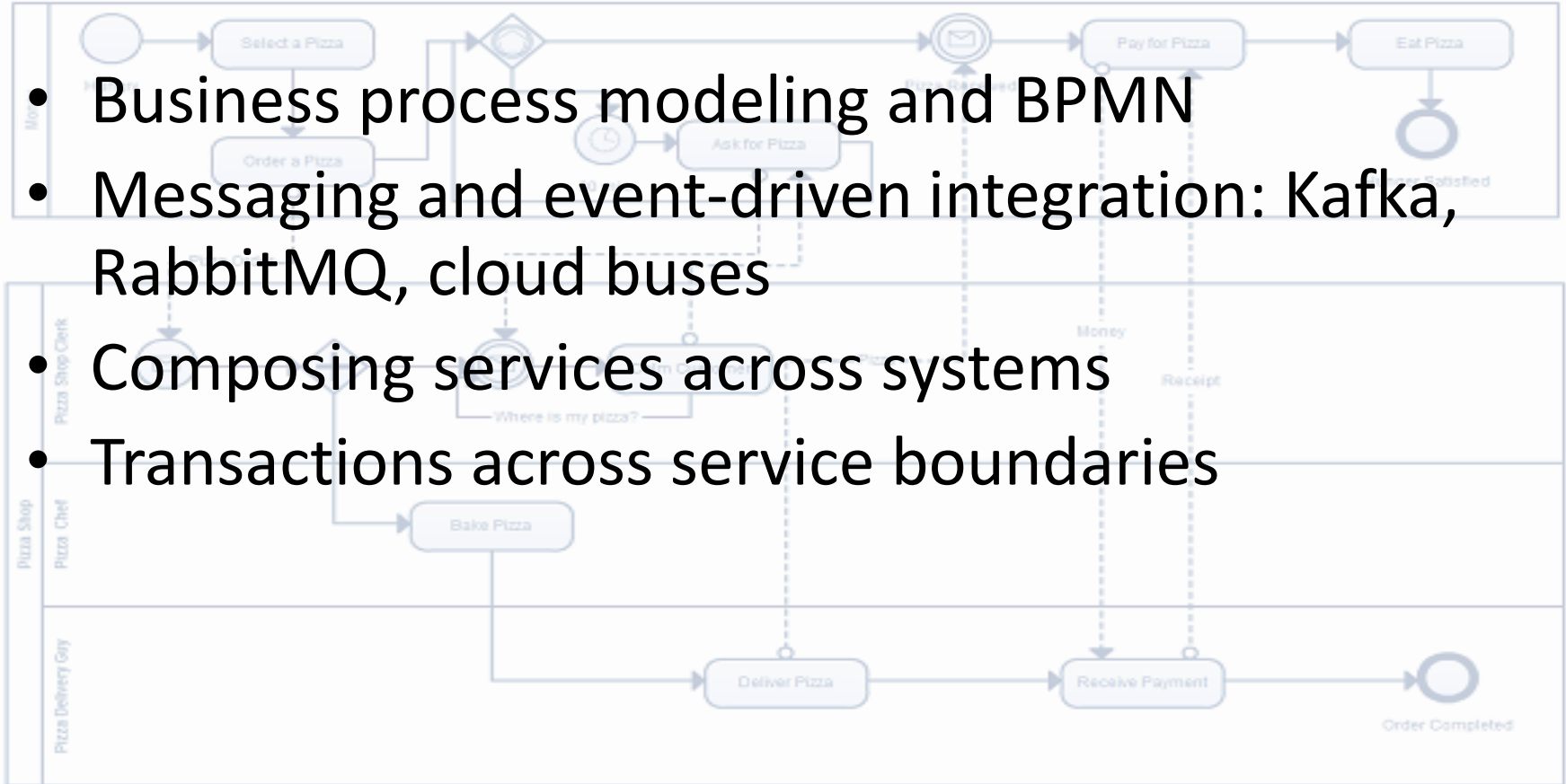
Golden paths, developer experience,
governance through code

#5 Enterprise Applications: ERP, SCM and CRM

- ERP, SCM and CRM: what these systems actually do
- SAP, Salesforce, Microsoft Dynamics
- Extension and integration; why replacing them is hard
- Workflow and orchestration systems

#6 Business Processes, Messaging and Service Composition

- Business process modeling and BPMN
- Messaging and event-driven integration: Kafka, RabbitMQ, cloud buses
- Composing services across systems
- Transactions across service boundaries



#7 Datastores, Databases and Query Languages

- Relational stores and SQL; NoSQL: key-value, document, graph, vector
- ACID, CAP theorem, consistency models
- Polyglot persistence — choosing a store against an access pattern
- Caching, replication, performance and scalability

#8 Data Engineering and the Modern Data Stack

RDBMS Vendors (SQL)

- MySQL/MariaDB
 - Oracle
 - PostGres
 - Informix
- ACID, CAP Theorem,
 - Performance & Scalability
 - Caching & Replication
 - Normalization
 - Security

Warehouses, lakes and lakehouses

ELT and transformation; batch versus streaming

Data quality, contracts and lineage

BI and analytics over the stack

Popular NoSQL databases

- Cassandra
- MongoDB
- Hbase
- Redis ([most popular key value store](#))
- BigTable/HyperTable
- Lucene/Solr
- Memcached (distributed memory caching)
- Neo4j (graph DB)

#9 Enterprise AI I — Platforms and MLOps

- Enterprise ML tooling and platforms
- Feature and model management
- Deployment, monitoring and drift
- MLOps in the enterprise

#10 Enterprise AI II — LLM Integration

- Retrieval-augmented generation over enterprise data
- Agents and tool use
- Evaluation, guardrails and governance
- The cost of inference at enterprise scale

#11 Enterprise Security, Identity and Supply Chain

- Identity, authentication and authorization; zero trust
- Software supply-chain integrity: SBOM, signing, provenance
- Attacks, attackers and secure development practice
- Compliance and audit

#12 Resilience, Observability and Monitoring

- Prometheus, ELK and Grafana
- SLOs and error budgets
- Incident response and the operational feedback loop
- Section 49: delivered asynchronously via Canvas (week 14)

Wrap-Up: Presentations and Final Exam

- Final Project Presentations — week 15 (Dec 1 / Dec 2)
- Final Exam Review
- Final Exam: Sec 03 Tue Dec 15 · Sec 49 Wed Dec 9 — 5:30-7:30 PM

BREAK

In-Class Project Team Formation

- Breakout Rooms Open
- Groups of 2-4 students
- Select a team leader
- Select a team name (also logo/mascot/etc..)
- Create a team code repository
- Submit a Word document for Project Team Formation (due 9PM PST)





HOMEWORK

Reading Assignment

- Fowler (Ch. 1)
- Newman (Ch. 1)
- [TheNewStack, Book 1](#), “DOCKER FUELS RETHINKING OF THE OPERATING SYSTEM” pp 87-97



Introduction to Ansible

What is Ansible?



- Ansible is an open source software that automates software provisioning, configuration management, and application deployment
- Ansible connects to hosts (target) via SSH, remote PowerShell or via other remote APIs.
- Ansible supports configuration management with examples as below.
 - Configuration of servers
 - Application deployment
 - Continuous testing of already install application
 - Provisioning
 - Orchestration
 - Automation of tasks

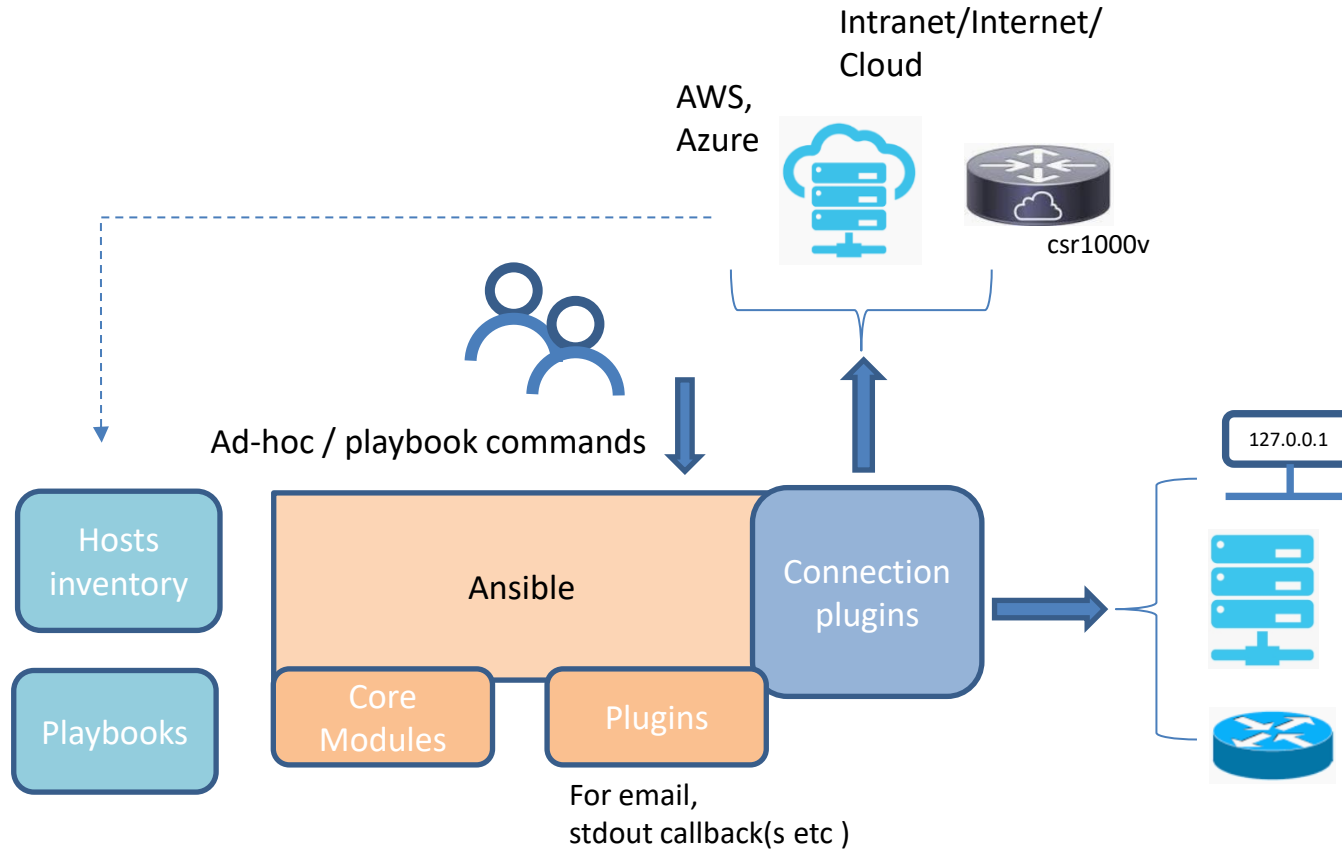
“Ansible is quite fun to use right away. As soon as you write five lines of code it works. With SSH and Ansible I can send commands to 500 servers without having even used the servers before.”

MARK MAAS UNIX/LINUX SYSTEMS
ADMINISTRATOR
BINCKBANK

Why Ansible?

- It is a free open source application
- Agent-less – No need for agent installation and management
- Intent based
- Python/YAML based
- Highly flexible and configuration management of systems.
- Extensive library of Modules for network management:
https://docs.ansible.com/ansible/2.8/modules/list_of_network_modules.html
- Custom modules can be added if needed
- Configuration roll-back in case of error
- Simple and human readable
- Self documenting

Components



Ansible use cases:

- Network automation with network module, to manage config networks devices
- With Jenkins module, implement continuous integration (orchestrate, provision, configure)
- Customizable, scalable security tasks with ssh and APIs (Cisco NX-OS nxapi, Netconf ...)
- Cloud provisioning and configuration with AWS, Azure, OpenStack ... modules
- Logging and monitoring APIs and modules (Grafana, Elastic Search, Splunk)

Windows / Mac / Linux

Windows: **No**



Ansible cannot run on a Windows host and can only manage Windows hosts, but Ansible can be [run under the Windows Subsystem for Linux \(WSL\)](#).

Mac: **Yes**



Install **Xcode**

```
$ sudo easy_install pip  
$ sudo pip install ansible --quiet
```

```
$ sudo pip install ansible --upgrade
```

Then, if you would like to update Ansible later, just do:



Linux (RHEL/CentOS)



- Ansible is not available in the default RHEL repositories
- Install Extra Packages for Enterprise Linux (EPEL)

```
sudo rpm -i epel-release-latest-7.noarch.rpm
```

- Then (after updating),

```
sudo yum install ansible
```

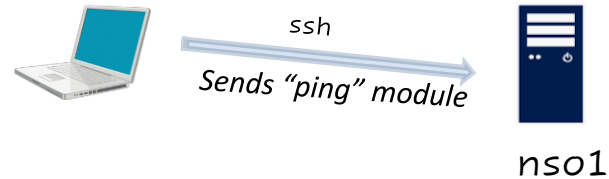
Howto: check connection with ping

Inventory file: hosts

```
[nso]  
nsol ansible_host=127.0.0.1 ansible_user=vagrant ansible_port=2222
```

Command:

```
$ ansible -m ping nsol  
nsol | SUCCESS => {  
  "ansible_facts": {  
    "discovered_interpreter_python":  
    "/usr/bin/python"  
  },  
  "changed": false,  
  "ping": "pong"  
}
```



Command verbose mode:

```
ansible -m ping nsol -vvv
```

Create the RSA Key Pair

- The first step is to create the key pair on the Server machine

```
$ ssh-keygen -t rsa
```

- Once you have entered the Gen Key command, you will get a few more questions:

- Enter file in which to save the key

- (~/ .ssh/id_rsa):

- Enter no password for the next prompt

- Copy the Public Key

```
ssh-copy-id <user@target-machine>
```

- Repeat the same process for other machines you wish to login automatically with.

Ansible does not expose a channel to allow communication between the user and the ssh process to accept a password manually to decrypt an ssh key when using the ssh connection plugin (which is the default). The use of ssh-agent is highly recommended.

- **ssh-agent** is a program to hold private keys used for public key authentication (RSA, DSA). The idea is that **ssh-agent** is started in the beginning of an X-session or a login session, and all other windows or programs are started as clients to the **ssh-agent** program.

```
function start_agent {
    echo "Initialising new SSH agent..."
    /usr/bin/ssh-agent | sed 's/^echo/#echo/' >
"${SSH_ENV}"
    echo succeeded
    chmod 600 "${SSH_ENV}"
    . "${SSH_ENV}" > /dev/null
    /usr/bin/ssh-add;
}

# Source SSH settings, if applicable

if [ -f "${SSH_ENV}" ]; then
    . "${SSH_ENV}" > /dev/null
    #ps ${SSH_AGENT_PID} doesn't work under cywgin
    ps -ef | grep ${SSH_AGENT_PID} | grep ssh-agent$ >
/dev/null || {
        start_agent;
    }
else
    start_agent;
fi
```

See also: <https://developer.github.com/v3/guides/using-ssh-agent-forwarding/>

Validating Ansible install

```
$ ansible localhost -m ping
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

Ad-hoc commands:

syntax :

```
ansible <pattern> -m <module_name> -a "<module options>"  
ansible-doc -t module setup
```

Example:

```
$ ansible ns01 --become -m service -a "name=firewalld state=stopped" --ask-become-pass  
BECOME password: xxxxxx  
      ns01 | SUCCESS => { "ansible_facts": { "discovered_interpreter_python":  
"/usr/bin/python" } }
```

Inventory file: hosts

```
[ns0]  
ns01 ansible_host=127.0.0.1 ansible_user=vagrant ansible_port=2222
```

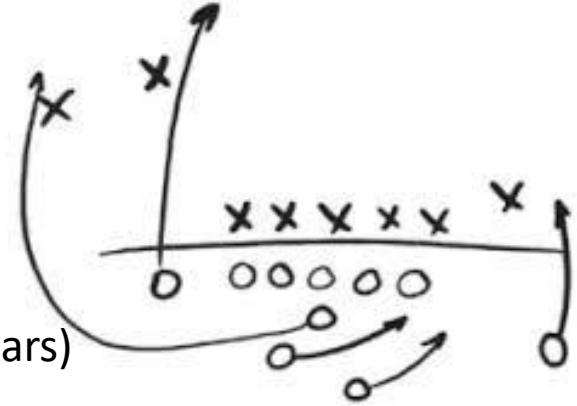
Playbook example

```
--- # file: webservers.yml
- hosts: webservers
  handlers:
    - name: reload apache2
      service: name=apache2 state=reloaded
  tasks:
    - name: Install Apache HTTP server
      apt: name=apache2 update_cache=yes
    - name: Install Apache Modules
      apache2_module: name={{ item }} state=present
      with_items:
        - proxy
        - proxy_httpd
      notify: reload apache2
  remote_user: deploy
  sudo: yes
```

Playbooks

What is a Playbook?

- Describes policies your managed machines shall enforce
- Consist of vars, tasks, handlers, files, templates and roles
- Expressed in the YAML format (dictionaries, lists and scalars)



BASIC

- Manage configurations of and deployments to remote machines

ADVANCED

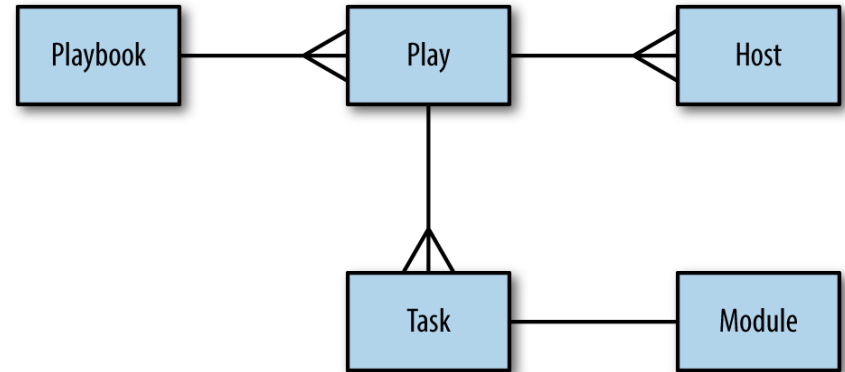
- Sequence multi-tier rollouts involving rolling updates, and can delegate actions to other hosts, interacting with monitoring servers and load balancers along the way

Playbook Concepts

- **Hosts and Users** – The specific machines in your infrastructure to target and details of the remote user for access
- **Task** – a specific, modular item of work that needs to be done
- **Play** – the set of hosts and tasks to be executed on those hosts
- **Modules** – a package of scripts for specific functionality (unix paradigm)
- **Variables** – Reference values that can be used in tasks or in template files. Any valid YAML value can be used, String, Boolean, List, Dictionary. Reference with `{{ braces }}` notation

Playbooks – Sections

- Playbook contains Plays
- Plays contain Tasks
- Tasks call Modules
- Tasks run Sequentially (asynchronous)
- Handlers are triggered by Tasks (run once at the end of Play)



Playbooks – Variables

- Variables

```
vars:
  key_file: /etc/nginx/ssl/nginx.key
  cert_file: /etc/nginx/ssl/nginx.crt
  conf_file: /etc/nginx/sites-available/default
  server_name: localhost
```

- Ansible also allows you to put variables into one or more files, using a section called *vars_files*

```
vars_files:
  - nginx.yml
```

- To set the value of a variable based on the result of a task, create a *registered variable* using the register clause when invoking a module

```
tasks:
- name: getting ip address of eth0
  shell: ip r 1 | grep -e eth0 | grep default | grep -oE "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"
  register: eth0
```

Playbooks – Facts

- *Facts* are information discovered from a *Host*

```
$ ansible -m setup localhost
localhost | SUCCESS => {
  "ansible_facts": {
    "ansible_all_ipv4_addresses": [
      "172.23.122.41"
    ],
    "ansible_all_ipv6_addresses": [
      "fe80::250:56ff:feab:ab41"
    ],
    "ansible_apparmor": {
      "status": "disabled"
    },
    "ansible_architecture": "x86_64",
    "ansible_bios_date": "10/22/2013",
    "ansible_bios_version": "6.00",
    "ansible_cmdline": {
      "KEYBOARDTYPE": "pc",
      "KEYTABLE": "us",
      "LANG": "en_US.UTF-8",
```

Privilege Escalation

- Ansible allows you to 'become' another user
 - Uses tools like: `sudo`, `su`, `pfexec`, `doas`, `pbrun`, `dzdo`, `ksu`, `runas`, `machinectl` and others
- Directives
 - **become**
set to yes to activate privilege escalation.
 - **become_user**
set to user with desired privileges — the user you become, NOT the user you login as.
Does NOT imply `become: yes`, to allow it to be set at host level.
 - **become_method**
(at play or task level) overrides the default method set in `ansible.cfg`

```
- name: Run a command as nobody
  command: somecommand
  become: yes
  become_method: su
  become_user: nobody
  become_flags: '-s /bin/sh'
```

Tasks and Handlers

```
- name: template configuration file
  template:
    src: template.j2
    dest: /etc/foo.conf
  notify:
    - restart memcached
    - restart apache
```

- Ansible modules should be idempotent and should relay when they have made a change on the remote system
- Ansible Playbooks recognize this and have a basic event system that can be used to respond to change
- Actions called **notify** are triggered at the end of each block of tasks in a play, and will **only be triggered once** even if notified by multiple different tasks – e.g. multiple resources may indicate that apache needs to be restarted because they have changed a config file, but apache will only be bounced once to avoid unnecessary restarts.
- The things listed in the notify section of a task are called handlers.
- Handlers are lists of tasks, not really any different from regular tasks, that are referenced by a globally unique name, and are notified by notifiers. If nothing notifies a handler, it will not run. Regardless of how many tasks notify a handler, it will run only once, after all of the tasks complete in a particular play.

Asynchronous Tasks

```
---  
- hosts: all  
  remote_user: root  
  
  tasks:  
    - name: simulate long running op (15 sec), wait for up to 45 sec, poll every 5 sec  
      command: /bin/sleep 15  
      async: 45  
      poll: 5
```

- By default tasks in playbooks block, meaning the connections stay open until the task is done on each node. This may not always be desirable, or you may be running operations that take longer than the SSH timeout
- To avoid blocking or timeout issues, you can use asynchronous mode to run all of your tasks at once and then poll until they are done
- To launch a task asynchronously, specify its maximum runtime and how frequently you would like to poll for status. The default poll value is 10 seconds if you do not specify a value for *poll*:

Magic Variables

- There are a few variables that define the meta information about the nodes, inventory, and plays, for example, which groups a node belongs to, what groups are part of the inventory, which nodes belong to which group, and so on. These variables, which are implicitly set, are called **magic** variables, and are very useful for discovering nodes and topology information
- The most important of these are **hostvars**, **group_names**, and **groups**. Users should not use these names themselves as they are reserved. **environment** is also reserved
- **hostvars** lets you ask about the variables of another host, including facts that have been gathered about that host. If, at this point, you haven't talked to that host yet in any play in the playbook or set of playbooks, you can still get the variables, but you will not be able to see the facts.

If your database server wants to use the value of a 'fact' from another node, or an inventory variable assigned to another node, it's easy to do so within a template or even an action line:

```
{{ hostvars['test.cisco.com']['ansible_distribution'] }}
```

commands: points to ponder

- Passing Variables via CLI

```
ansible-playbook playbook.yml -e "USERNAME=user001"
```

- Check mode (Simulation – it will not make any changes on remote systems)

```
ansible-playbook playbooks/PLAYBOOK_NAME.yml --check
```

- Diff mode (reports changes made or, if used with `--check`, the changes that would have been made.

```
ansible-playbook playbooks/PLAYBOOK_NAME.yml --check --diff
```

- `--syntax-check` option, checks only syntax

```
ansible-playbook playbooks/PLAYBOOK_NAME.yml --syntax-check
```

- Copy file from Ansible server to target machines using the copy module

```
ansible -m copy -a "src=/tmp/file dest=/tmp/filetarget;" all
```

roles: modularization / reuse

- Primary mechanism to break playbook into set of files
- Simplifies writing complex LONG playbooks
- Set of behavior we apply to a host or set of hosts

Examples: database role, webserver role, firewall role

Typical Firewall role:

- `roles/firewall/tasks/main.yml`

Tasks

- `roles/firewall/templates`
Holds jinja2 template files (example: POLICIES, RULES etc)
- `roles/firewall/handlers/main.yml`

Handlers

- `roles/firewall/vars`
Variables that should not be overridden
- `roles/firewall/files`
Holds files to be uploaded to hosts (example: script files)

Word to playbooks ?

Word document instructions:

2.3	NSO os upgrade package usage
2.3.1	Pre-configuration
2.3.2	Staging Images to devices
2.3.3	Os upgrade
2.3.4	Troubleshooting os upgrade.....

2.3.2.1 Cisco ios xr 32-bit

2.3.2.1.1 Only one device

- o To perform staging, load below service configurations into NSO a
 1. Os-upgrade service configuration.
 2. Os-upgrade-service service configuration.
 3. Execute the action.

- o Login into NSO cli using command:

✓ `nso cli -u <nso_username> -C`

```
os-upgrade-service properties sample-file Pi256.bin
os-upgrade-service properties transfer-threshold-time 18188000
os-upgrade-service lookup-data upgradePathLookup image-version-mapping cisco-ios-xr
    entries ASR9K 6.6.2 6.2.3 ←
!
!
....!
os-upgrade-service lookup-data imageServerDetails image-servers asia
server 10.122.32.188 ←
    username    robot ←
    password    secretpassword ←
    protocol    scp ←
    server-root-dir /chroots/robot/623 ←
!
!
os-upgrade-service lookup-data version-image-lookup image-version-mapping cisco-ios-xr
entries ASR9K 6.2.3 ←
```

Os-upgrade: playbooks and roles.

```
---
- name: "playbook: insall prereq scripts"
  hosts: tsa-ops-01
  tasks:
    - name: Load variables
      include_vars:
        file: nso.yaml
        name: nso
      tags:
        - always
  roles:
    - { role: pre-requisites }
```

```
✓ roles
  ✓ pre-requisites
    ✓ tasks
      ! create-prerequisite-commands.yaml
      ! main.yaml
    ✓ templates
      ❗ add2-nso-user-groups.sh.j2
      ❗ do-staging32-step1.sh.j2
      ❗ do-staging32-step2.sh.j2
      ❗ do-upgrade32-setup-upgrade-rules.sh.j2
      ❗ do-upgrade32-setup-upgrade-service.sh.j2
      ❗ set-device-global-config.sh.j2
    ✓ vars
      ! main.yaml
```

```
---
- import_tasks:
  create-prerequisite-commands.yaml
```

```
---
- name: create-prereq scripts at /tmp
  template:
    src: "{{ item.src }}"
    dest: "{{ item.dest }}"
    mode: 775
    owner: "{{ ansible_user }}"
    group: ncsadmin
  with_items:
    - { src: "add2-nso-user-groups.sh.j2" , dest: "/tmp/add2-nso-user-groups.sh" }
    - { src: "set-device-global-config.sh.j2" , dest: "/tmp/set-global-config.sh" }
    - { src: "do-staging32-step1.sh.j2" , dest: "/tmp/do-staging32-step1.sh" }
```

Homework #1 – Ansible

- Configure two VMs, **VM1** and **VM2** either on your own hardware, or in a cloud environment. Configure Ansible to deploy a webserver on **VM1** and **VM2** on port 8080 with a web page that is accessible from a web browser, and displays the message: “Hello World from SJSU-X” where X is 1 or 2 depending on which webserver instance, VM1 or VM2.
- Include in the Ansible playbook, plays to **deploy** and **undeploy** the webserver resources
- Submit a Word document, with screenshots showing your work, and a demo, and all ansible code/scripts via github

SAN JOSÉ STATE UNIVERSITY *powering* SILICON VALLEY

TOGAF is a framework for developing an enterprise architecture. It consists of a detailed method and a set of supporting tools

